

ElixirLabs · [Follow publication](#)

Event Bus Implementation(s)

5 min read · Jul 24, 2017



Mustafa Turan

Follow



Listen



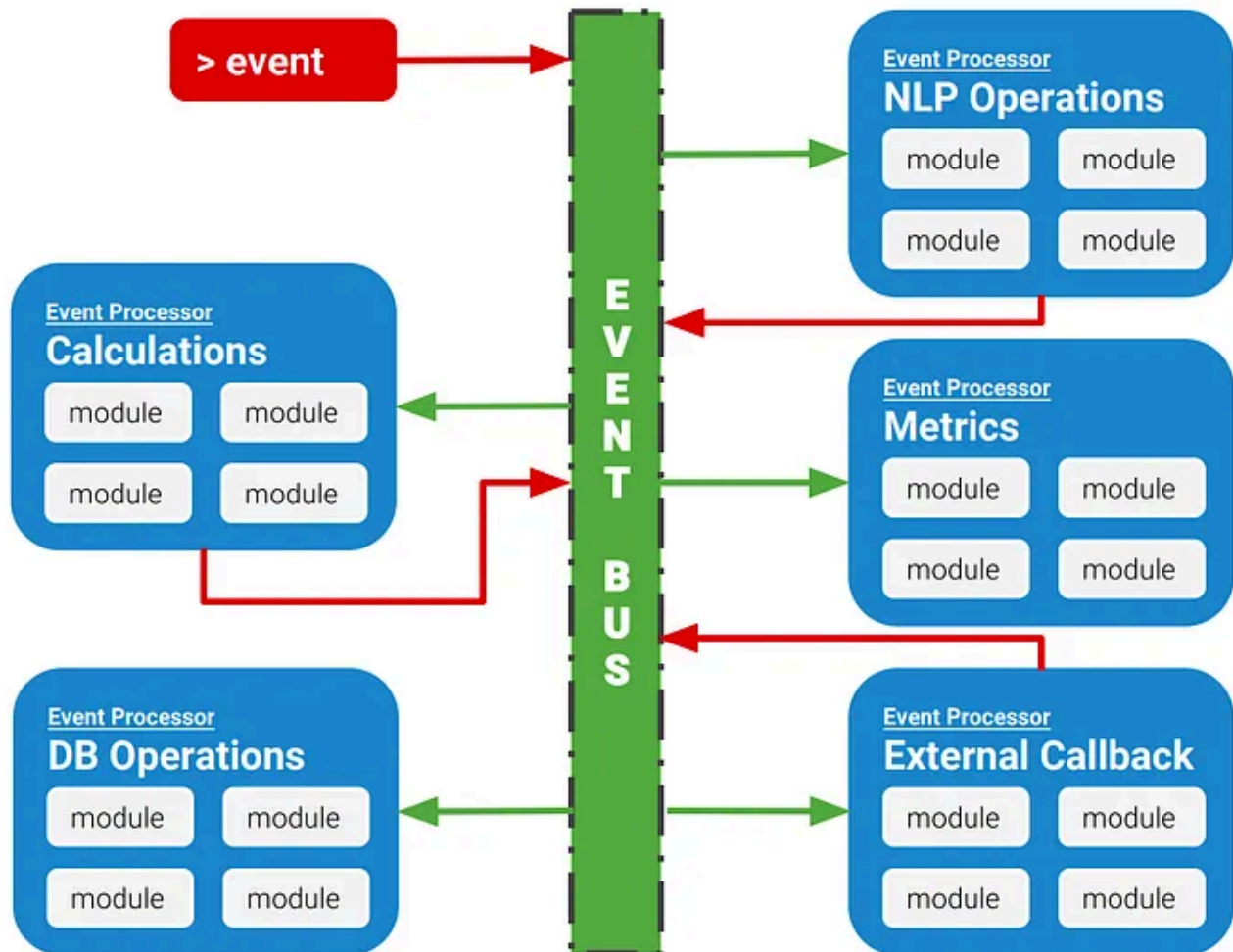
Share



More

Event-driven architecture pattern is a distributed asynchronous architecture pattern to create highly scalable reactive applications. The pattern suits for every level application stack from small to complex ones. The main idea is delivering and processing events asynchronously.

There are a couple of ways to implement event-driven pattern to your stack. One of them is using “**Broker Topology**”. In broker topology, you send events to a central broker and all subscribers of these broker receive and process events asynchronously.

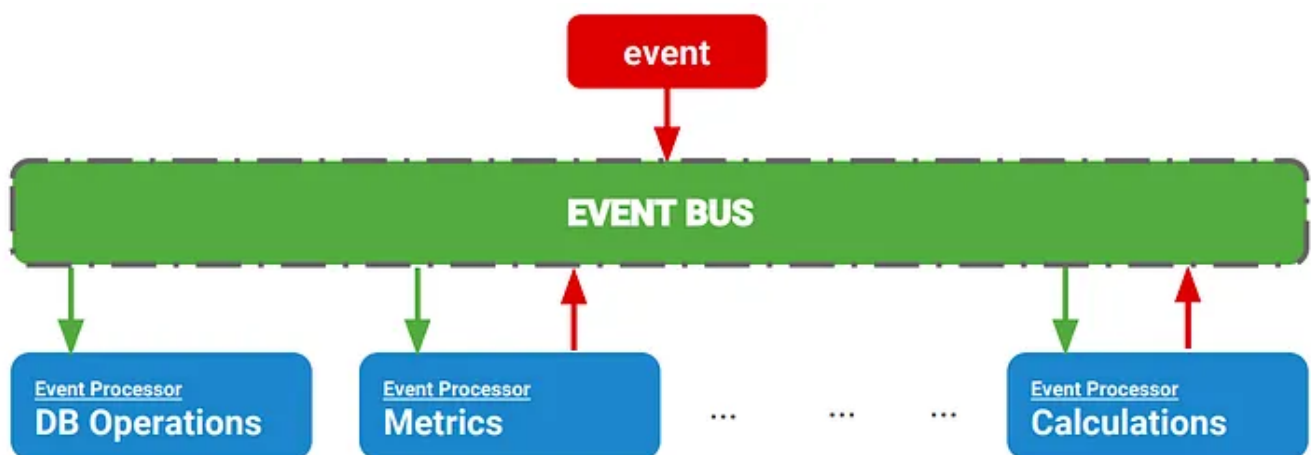


@mustafaturan

Event Bus with multiple subscribers(green arrows) and notifiers(red arrows)

In this story, I shared *four implementations* of event bus and with their *goods* and *drawbacks*. In all implementations, all event generators send events to the event bus. Event processors(subscribers) subscribe to the event bus. And subscribers can also create new events and pass it to the event bus.

i) Notify the Event to All Subscribers



@mustafaturan

Event Bus sends all incoming events(red arrows) to all subscribers(blue boxes) without any touch

Actors

Event Bus, Subscribers (Event Processor), Event Creators

Implementation

Whenever the event bus receives an event from event creators, it passes the event to all subscribers. Subscribers process the event data with their own logic. Subscribers can also create new events and pass it to the event bus too. Event bus does not care about if the receiver fails to receive the event message.

Functions Needed

Notify, subscribe, unsubscribe, subscribers are enough to implement such event bus.

Goods

It is very simple in terms of development, not too much responsibility.

Drawbacks

In this approach, you are replicating the event data to all subscribers, even if the subscribers do not have the power to consume this event data. This means that your memory demand may increase with the multiplication of the subscribers in time.

It also does not care about the assurance of the delivery. It tries to send the event to the subscriber only once but it does not care about the broken pipes, connection errors and other possible lacks.

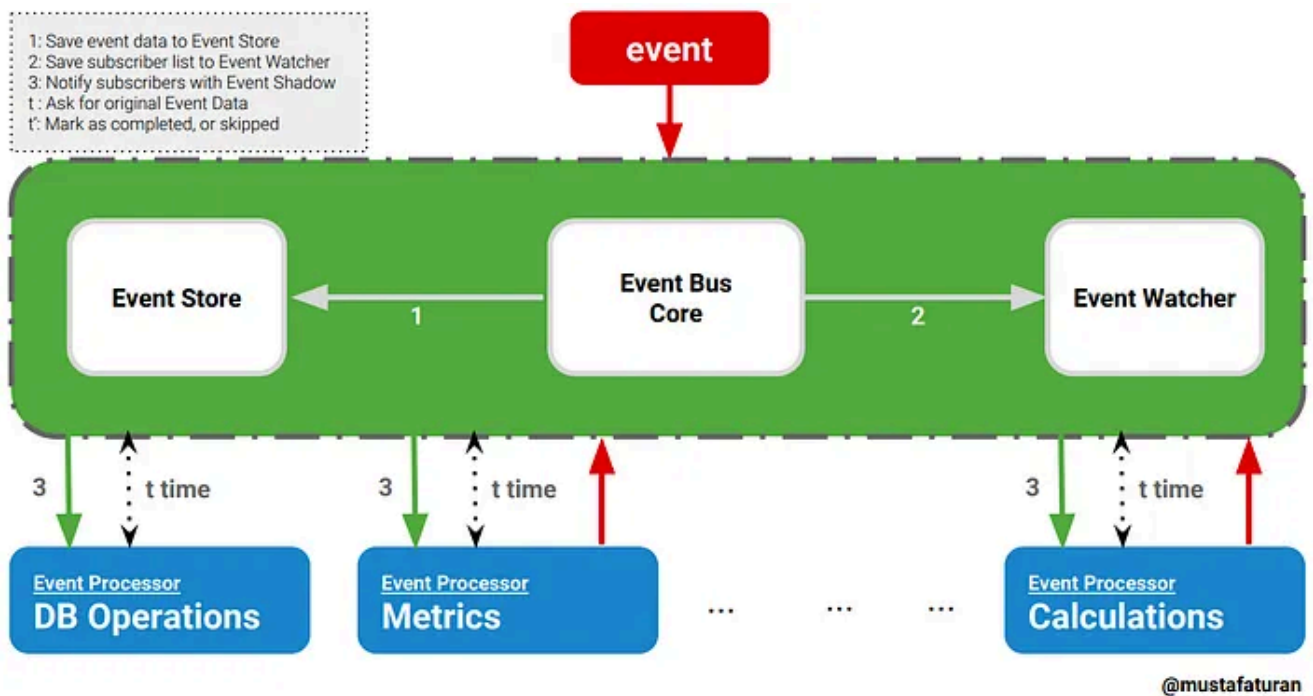
This implementation notifies all subscriber without filtering. So, filtering should be done in subscribers.

On JS front-end world `reduxjs` library may be given as a sample of this implementation. When an event dispatched by `actions`, all `reducers` receive the same event type and event data and decides what to change.

Such implementation in Elixir lang:

https://github.com/mustafaturan/event_bus/commit/08a955294c1d1c5bec2c7fef28ed255400e5f322

i2) Notify the Event Shadow to All Subscribers



Event Bus with Event Store and Event Watcher

Actors

Event Bus, Event Creators, Subscribers, Event Store, Event Watcher

What is Event Shadow?

Probably, you never heard this term before, do not worry I made it up. Event shadow is basically a reference data to your original event data.

Implementation

Whenever the event bus receives any event, it saves it to an **Event Store**. Then passes the event shadow to all subscribers. Subscribers process the event shadow and fetch the event data from Event Store when they start processing.



When an event comes to the event bus, it creates a watcher with the list of subscribers. **Event Watcher** is responsible for deletion of event data from the Event Store when all subscribers process the event.

Event bus does not care about if the receiver fails to receive the event message. But it automatically marks the Event Watcher as '**skipped**' when any subscriber fails to

receive event shadow.

Functions Needed

Notify, subscribe, unsubscribe, subscribers, save/delete/fetch_event_data, mark_as_completed/skipped are enough to implement such an event bus.

Goods

This approach comes with less memory consumption. It is possible to implement one more function to watch the current status of a specific event or all events.

Drawbacks

In this approach, each subscriber needs to call **fetch_event_data**, **mark_as_completed**, and **mark_as_skipped** functions. You need to implement a **read-heavy event store**. And you need to implement also an 'event watcher' to delete the event data when all subscribers processed the data. And sadly, event store write persistence should be blocking.

Like in the implementation **i1**, this implementation notify all subscriber without filtering. So, filtering should be done in subscribers. Which is not too bad, but if you are using the event bus over networks, this will cause unnecessary network traffic.

Such implementation in Elixir lang:

https://github.com/mustafaturan/event_bus/releases/tag/v0.2.1

i3) Notify the Event Shadow to Filtered Subscribers

This implementation is same as the **i2** but it is filtering the events before notifying the subscribers. In this approach, you need to register subscribers with their interests of topics. Note: It is good to have a regex filter on topics(event types) like in RabbitMQ.

It has the same goods and drawbacks as in the implementation **i2**, except **i3** supports filtering on events.

Such implementation in Elixir lang: https://github.com/mustafaturan/event_bus

i4) Ordered Delivery to Subscribers

To guarantee the ordered event delivery, the idea is to partition your data before passing to interested in subscribers. The [Kafka paper](#) is a great way to start such an implementation. In basic, the producer decides for the event partition with a custom function like ``CRC32``. And only one consumer is responsible only for that partition.

(Addition at 28 July 2017) In Kafka, consumers are subscribing to the given list of topics to get dynamically assigned partitions and then they poll the data from the topics or partitions specified using one of the `subscribe/assign` APIs.

Drawbacks

Increasing, decreasing the number of partition might be a pain. You need to implement your own partition function on the producer. The consumer implementation will be as hard as broker implementation.

Notes on Subscriber Implementations

In i1 and i2 implementations, all subscribers should match the event type(topic) first in a **blocking** manner to *prevent unnecessary stacks in memory*.

In i3 and i4 implementations, you should match event type(topic) in a **non-blocking** asynchronous way because only the subscribed topic data comes to the event processors.

In general, all subscribers should process the event asynchronously without any blocking manner. Also, subscribers can create new events and pass those events to the event bus.

Open in app ↗

≡ Medium



coming with different promises. For example, Kafka promises on the 'guaranteed and ordered delivery', AMQP promises on the 'guaranteed delivery' and ZeroMQ comes with patterns to develop your own custom broker.

If you are using Elixir or Erlang for the development, try the [event bus](#) library which is an implementation of the i3.

GitHub - otobus/event_bus: Traceable, extendable and minimalist **event bus implementation for...**

surfer: Traceable, extendable and minimalist **event bus** implementation for Elixir with built-in **event store** and...

github.com

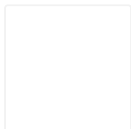
Eventbus +

Event Sourcing +

Software Architecture +

Elixir +

Microservices +



Follow

Published in ElixirLabs

859 followers · Last published Jul 16, 2018

Elixir, OTP, Phoenix Framework, Ecto



Follow

Written by Mustafa Turan

612 followers · 67 following

Go, Elixir, Ruby, Software Architecture, Microservices

Responses (3)



Sakharwsejal

What are your thoughts?



Vikas Sood
May 23, 2018



Great post!! I used a similar design in implementing an event bus for my websocket server I wrote.



6

[Reply](#)



Manuel Lucas
Mar 30, 2019



Great post!!



1

[Reply](#)



Hubert
Jul 25, 2023 (edited)



Thanks for sharing.

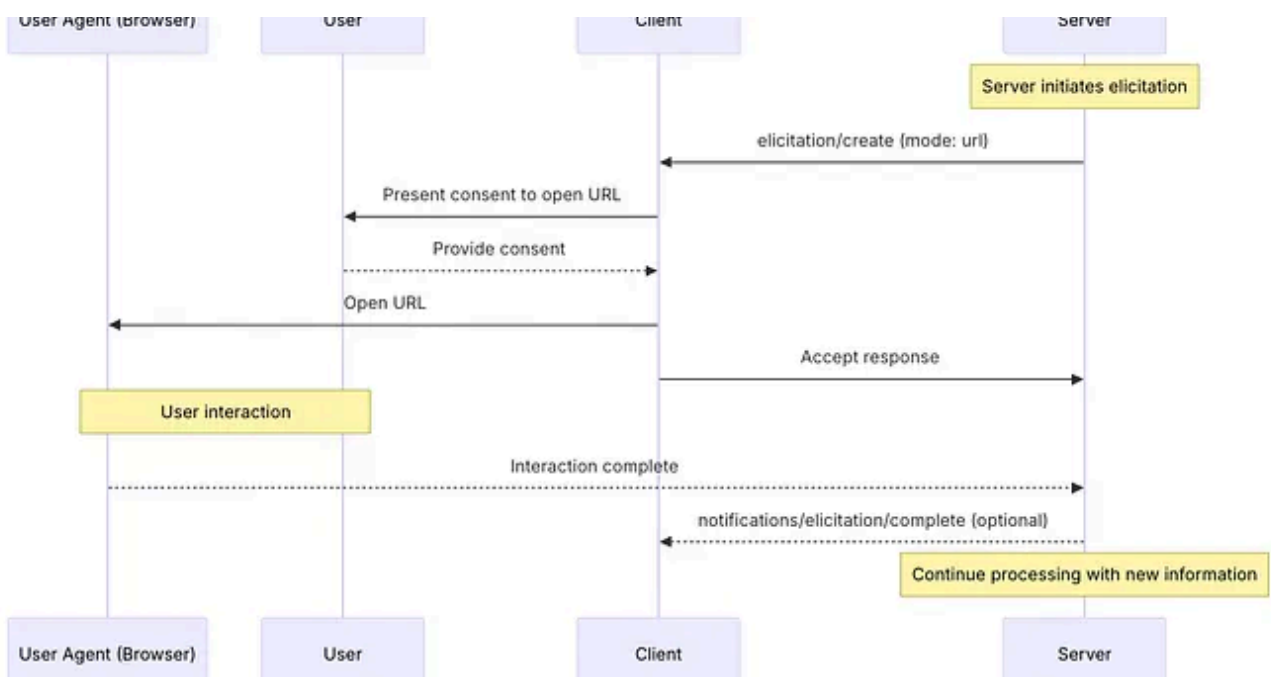
Should we add retry mechanism to handle event to protect the event bus of server is downtime?



1 reply

[Reply](#)

More from Mustafa Turan and ElixirLabs

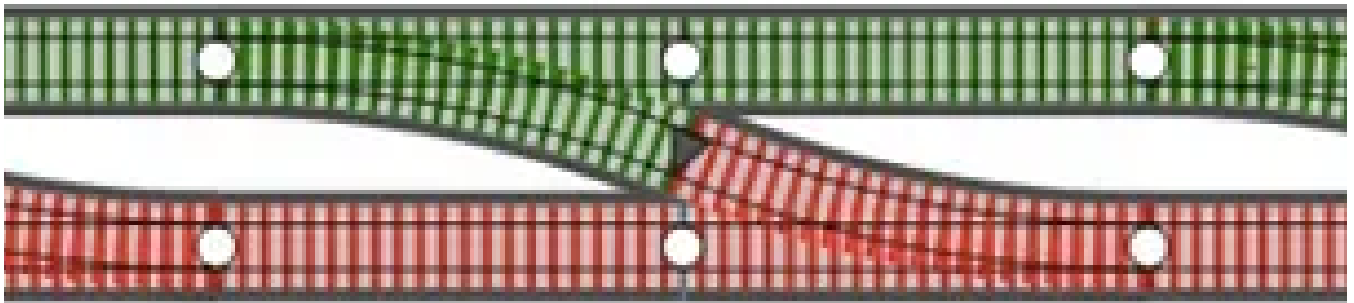


MCP Authentication & Authorization Pain Points

The Model Context Protocol (MCP) promises a standardized way for AI models to interface with the world—a universal translator between...



UpdateDb



 In ElixirLabs by Mustafa Turan · Feb 26, 2017

Railway Oriented Programming in Elixir with Pattern Matching on Function Level and Pipelining

Coding seems cool without error checks, does not it? When if/else checks effect the next execution behaviours, then the code becomes a...

 213  7



EventBus

In ElixirLabs by Mustafa Turan · Jul 16, 2018

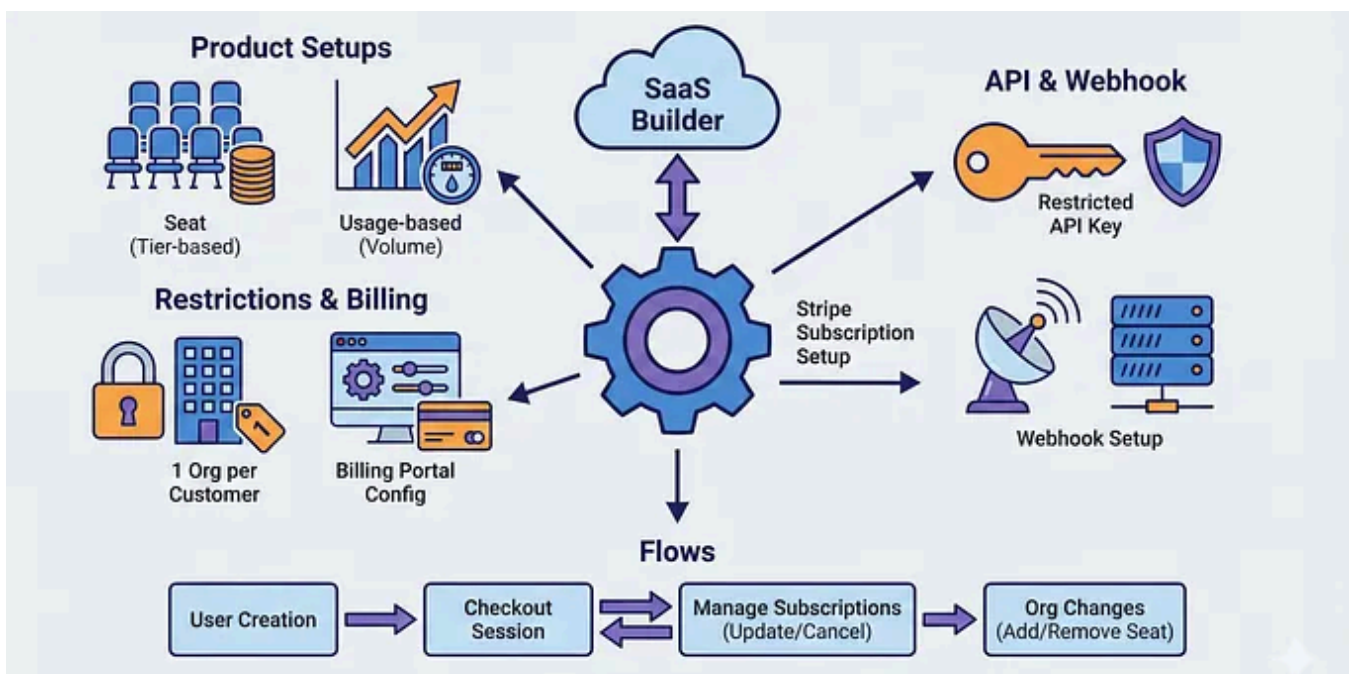
Decoupled Modules with Elixir EventBus

Elixir EventBus is a library that allows different modules to communicate with each other without knowing about each other. A...

812 5



...



Mustafa Turan · Dec 19, 2025

Stripe Subscription Notes from a first time SaaS Builder

I am bootstrapping my own SaaS product called HasMCP and can say one of the challenging parts is subscription management. Once you build...



See all from Mustafa Turan

See all from ElixirLabs
